

CSCI 4160/6963, ECSE 4965/6965

Reinforcement Learning

Homework 10

Overview

In this homework, you will implement one of the most popular deep reinforcement learning (RL) algorithms, namely double deep Q-learning, DDQN. You will work with two environments; (1) the Cartpole benchmark in OpenAI as a warmup; (2) the classic Atari Pong game, emulated in the Atari Learning Environments (ALE) library.

Cartpole is essentially a mix of mountain car and the inverted pendulum – you control a little cart that needs to balance a pole on top. You only have two actions, left and right. You get a reward of 1 after each step – if the pole drops below a certain degree, the episode is terminated.

In Pong, you control a paddle at the right end of the screen, whereas the opponent (the [old-school] AI) controls a paddle at the left end. Each player's goal is to hit a ball past the opponent. Although in theory there are three actions (up, down, noop), ALE tries to be true to original arcade game as much as possible, so there are three additional actions: fire, up+fire and down+fire, where the fire action is used in the arcade game to start the game (models the button in a classic arcade game). In ALE Pong, the actions up and up+fire do the same thing – so having 6 actions total is redundant – it is up to you whether you want to simplify the task by ignoring three of the actions (here's the official documentation if you wish to do so: <https://ale.farama.org/environments/pong/>). You get a reward of 1 for each point and a reward of -1 for each opponent point. The game is played until either player reaches 21.

Although ALE Pong is meant to be an image-based control task, we are going to simplify it by extracting the four-dimensional game state from the image (e.g., y position of both paddles, as well as x-y position of the ball). Similar to the Pendulum environment, I have provided a function, `get_state_from_image`, to get the Pong state from observations. You shouldn't have to modify that function.

In both tasks, your goal is to maximize the discounted return, which is equivalent to 1) keeping the pole upright as long as possible or 2) winning the game of Pong.

Logistics

All coding assignments in this course will be in Python. If you need help with Python, please talk to me ahead of time, so we can discuss the best way to get familiar with it.

For this assignment, you will be using your own computer, which should be sufficient. The code should not require more than 20 minutes to run on any standard laptop. You should feel free to use the CCI cluster as well if needed.

You are provided with skeleton starting code. Please use the Python libraries in the `requirements.txt` file that's provided on LMS. **Important:** I have updated the original `requirements.txt` file on LMS to include additional ALE packages.

Grading

The first main challenge in this assignment is to implement a stable DDQN algorithm with a target network, using a fully connected neural network, as provided in the `model.py` file. Note that the experience replay buffer is implemented in the `utils.py` file. Feel free to change that code if you wish to make any changes to the buffer functionality.

In both tasks, you are responsible for implementing the functions `select_action`, `eval_policy` and `train_policy`, as well as the main training loop.

In the case of Cartpole, the target undiscounted reward (as averaged over 10 simulator runs) is 200. You are allowed a total of 100K environment steps. You will lose 5 points for each 5 points of reward below the target.

In Pong, during training we will focus on playing a single round of the game (the original game is played until either player scores 21 points). During training, each episode will terminate with a reward of either 1 or -1. However, during evaluation, in your `eval_policy` function, you should play the game until 21. For a full grade, you need to achieve an undiscounted reward of at least 1, i.e., win the game (as averaged over 10 simulator runs). You are allowed a total of 200K environment steps. You will lose 5 points for each point of reward below the target.

Make sure your code prints out some indication of progress as well as all required information – we should not have to insert any print statements in order to find out whether your code works!

Finally, please provide answers to each of the questions below:

1) Why are we focusing on a single round of Pong during the training phase? In other words, suppose the score is 0:0 vs 10:0 in your favor (and suppose in both cases you win 21:0). What values will your Q network learn for the starting state in each of those two cases?

- 2) What changes did you have to make in Pong in addition to your Cartpole approach?
- 3) **[Graduate students only]** Give two reasons why the Pong environment is not a Markov Decision Process.

Hints and Tips

- Feel free to visualize both environments before you get started in order to familiarize yourself with the control tasks.
- DQN has a large number of hyperparameters. Feel free to change them as you see fit.
- You should evaluate your policy frequently (e.g., every 1000 or 10000 steps), and you can terminate your code once you have reached the target reward in the evaluation.
- You can use any regression loss, for deep Q-learning, we have discussed in class.
- You will most certainly need to implement target networks (that's what the `tau/target_update_freq` hyper-parameters are for). To do this in place, use the `copy_` function (e.g., loop through your policy's parameters using `for param, target_param in zip(policy.parameters(), target_policy.parameters())` and then call `copy_` on each `target_param`).
- Think about what information is provided by one observation in Pong.
- Start early. Deep RL is no joke.

Submission

Please use LMS to submit a zip file containing 1) your **.py** code (in four files, as provided), along with instructions on how to run it, 2) a **.pdf** file containing your answers to the above questions. The deadline is **11:59 pm, December 12**. This is the last day of class, so **you are not allowed to use an extension for this assignment**.